

SEGURANÇA DE SISTEMAS COMPUTACIONAIS, 2026/2

Avaliação de Vulnerabilidades em Aplicação Web

Cenário A, OWASP Juice Shop

EQUIPE

Diego Fonseca da Silva
Leonardo Pacheco B Dias
Gustavo Gonçalves Trindade
João Victor Rodrigues Santos

PROFESSOR

Ramicés dos Santos Silva

INSTITUIÇÃO

UNIVALI, Escola Politécnica, Ciência da Computação

DATA

14/09/2026

ROTEIRO

Sumário

Clique em qualquer item para navegar.

BLOCO A, CONTEXTO E AMBIENTE

01 Capa

03 Escopo e conduta ética

04 Ambiente Docker

05 Arquitetura e ferramentas

BLOCO B, ACHADOS

06 Visão geral dos sete achados

07 V1, SQL Injection no login (A03)

08 V2, hash MD5 exposto no JWT (A02)

09 V3, IDOR na cesta de compras (A01)

10 V4, exposição no erro e CSP ausente (A05)

11 V5, listagem e documento público (A05)

12 V6, login após 15 erros consecutivos (A07)

13 V7, recuperação por resposta fraca (A04)

BLOCO C, SÍNTESE

14 Cobertura OWASP

15 Priorização e correções

16 Conclusão

Escopo e conduta ética

Teste conduzido apenas contra a instância local do próprio grupo.

Alvo controlado

O OWASP Juice Shop é uma aplicação web propositalmente vulnerável, mantida pela comunidade OWASP para fins educacionais. Toda a exploração ocorreu em contêiner Docker acessível apenas em `127.0.0.1`, sem exposição à rede externa.

Base legal

Nenhum teste foi realizado contra sistema de terceiros. O roteiro exige autorização e escopo local. A equipe executou os testes apenas no próprio laboratório, conforme a conduta profissional exigida.

Segurança não é ataque gratuito. O objetivo foi encontrar e comunicar falhas para que possam ser corrigidas, a postura de um profissional responsável.

REPRODUTIBILIDADE

Ambiente em Docker

Um único serviço, porta ligada só ao host, imagem com tag fixa.

```
docker compose up -d      # sobe o Juice Shop
# acesso: http://127.0.0.1:3000
docker compose down       # derruba o ambiente
```

A porta do contêiner é ligada exclusivamente em `127.0.0.1`, de modo que a aplicação não fica acessível a partir de outro dispositivo da rede.

v20.2.0

Tag versionada reduz variações. O digest está registrado no README: o Compose usa a tag.

healthy

`healthcheck` configurado; `docker compose ps` confirma quando a app está pronta.

.env

Porta de host configurável, evita conflito sem editar o compose.

down/up

Sem volume persistente: down remove o contêiner; up cria outro a partir da imagem.

Arquitetura e ferramentas

Um contêiner, rede bridge isolada, host local como máquina de teste.

```
[ Navegador + Postman ] --HTTP--> [ 127.0.0.1:3000 ]  
    host local                      contêiner juice-shop  
                                   rede labnet (172.19.0.0/16)
```

Item	Valor
Alvo	http://127.0.0.1:3000
Rede Docker	labnet (bridge, isolada)
Sub-rede observada	172.19.0.0/16 (não fixada no Compose)

Ferramentas de apoio

[Docker Compose](#)[DevTools](#)[Postman](#)[jwt.io](#)[md5sum](#)[sqlmap](#)[OWASP ZAP](#)[curl](#)[Scripts Python](#)

O método combina reprodução, evidência HTTP, classificação OWASP/CWE e pontuação CVSS. Python reproduz V1, V2, V3, V6 e V7; V4 e V5 usam curl/Postman. ZAP baseline é citado no relatório; seu log não foi preservado entre os artefatos disponíveis.

RESULTADOS

Sete achados confirmados

Seis categorias distintas do OWASP Top 10:2021, cobertura mínima do roteiro atingida.

7

vulnerabilidades confirmadas

6

categorias OWASP cobertas

2

críticas (CVSS 9.1)

1

alta (CVSS 7.5)

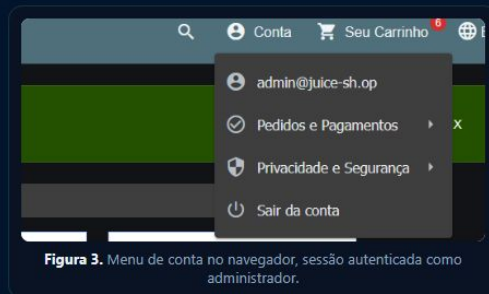
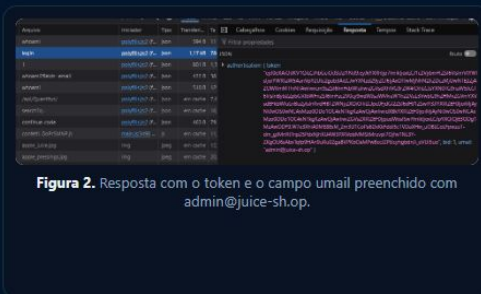
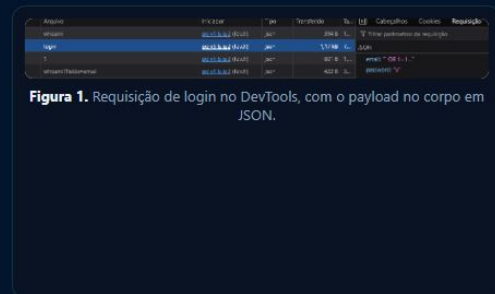
ID	Achado	OWASP	CWE	Severidade
V1	SQL Injection no login	A03	CWE-89	9.1 Crítico
V2	Hash MD5 da senha exposto no JWT	A02	CWE-522/916	6.5 Médio *
V3	IDOR na cesta de compras	A01	CWE-639	6.5 Médio
V4	Erro verboso; CSP ausente	A05	CWE-209/16	5.3 Médio
V5	Listagem de diretório em /ftp/	A05	CWE-548	7.5 Alto
V6	Login sem bloqueio contra força bruta	A07	CWE-307	6.5 Médio
V7	Reset de senha por pergunta de segurança	A04	CWE-640	9.1 Crítico

* Pontuações adotadas no relatório. V2: 6.5 no cenário autenticado; 7.5 é a leitura alternativa registrada com PR:N, não o score total da cadeia V1–V2. V6 e V7 têm limites de evidência explicados adiante.

SQL Injection no login

● CVSS 9.1 Crítico

O campo de e-mail aceita o payload `' OR 1=1--`, que comenta a verificação de senha e autentica o requisitante como o primeiro registro da tabela, a própria conta administrativa.

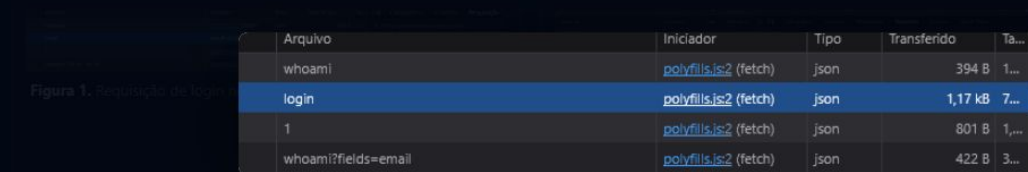


Confirmado de forma independente com sqlmap, que identificou a injeção booleana cega no parâmetro `email` e o SGBD SQLite. Reprodutível com `scripts/v1_sql_login.py`.

SQL Injection no login

● CVSS 9.3 Crítico

O campo de e-mail aceita o payload `' OR 1=1--`, que comenta a verificação de senha e autentica o requisitante como o primeiro registro da tabela, a própria conta administrativa.



Arquivo	Iniciador	Tipo	Transferido	Ta...
whoami	polyfills.js2 (fetch)	json	394 B	1...
login	polyfills.js2 (fetch)	json	1,17 kB	7...
1	polyfills.js2 (fetch)	json	801 B	1...
whoami?fields=email	polyfills.js2 (fetch)	json	422 B	3...

Figura 1. Requisição de login

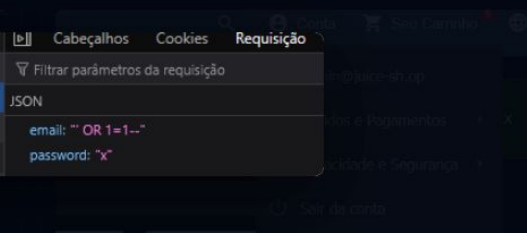


Figura 3. Menu de conta no navegador, sessão autenticada como administrador.

Confirmado de forma independente com sqlmap, que identificou a injeção booleana cega no parâmetro `email` e o SGBD SQLite. Reprodutível com `scripts/v1_sql3_login.py`.

SQL Injection no login

● CVSS 3.1 Crítico

O campo de e-mail aceita o payload `' OR '1'='1`, que comenta a verificação de senha e autentica o requisitante como o primeiro registro da tabela, a própria conta administrativa.

```
POST /api/login HTTP/1.1
Host: juice-sh.op
Content-Type: application/json
Content-Length: 100

{"email": "admin@juice-sh.op", "password": "1234567890"}
{"email": "admin@juice-sh.op", "password": "1234567890"}
```

Figura 1. Requisição de login no DevTools, com o payload no corpo JSON.

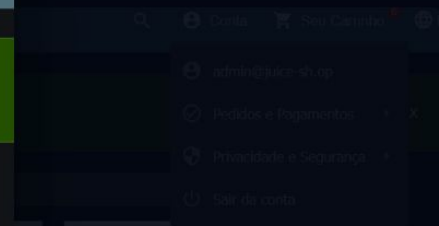
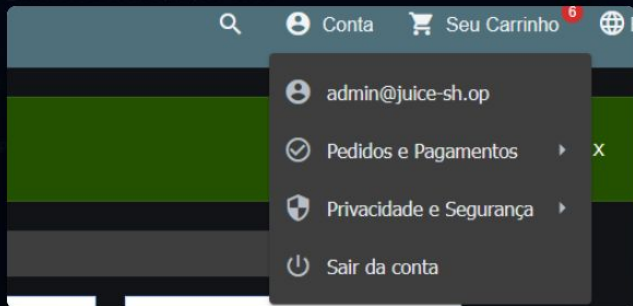
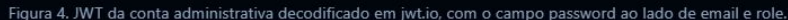


Figura 3. Menu de conta no navegador, sessão autenticada como administrador.

Confirmado de forma independente com sqlmap, que identificou a injeção booleana cega no parâmetro `email` e o SGBD SQLite. Reprodutível com `!scripts/v1_vuln_login.py`.



Controle de acesso quebrado na cesta

● CVSS 6.5 Médio

Uma conta autenticada lê a cesta de outra apenas trocando o identificador na URL. O servidor não verifica se o token pertence ao dono do recurso.

```
fetch("/rest/basket/2", {
  headers:{ Authorization:"Bearer "+token }
}).then(r=>r.json()).then(console.log)
// deveria responder 403, responde 200
```

A prova de conceito registra duas contas isoladas e usa o token da segunda para ler a cesta da primeira, sem depender de sessão logada manualmente.

```
Object { status: "success", data: { ... } }
  ↳ data: Object { id: 2, UserId: 2, createdAt: "2026-09-04T13:48:59.480Z", ... }
    ↳ Products: Array [ { ... } ]
      ↳ 0: Object { id: 4, name: "Raspberry Juice (1000ml)", description: "Made from blended Raspberry PI, water and sugar.", ... }
        ↳ length: 1
        ↳ <prototype>: Array []
        ↳ UserId: 2
        ↳ coupon: null
        ↳ createdAt: "2026-09-04T13:48:59.480Z"
        ↳ id: 2
        ↳ updatedAt: "2026-09-04T13:48:59.480Z"
      ↳ <prototype>: Object { ... }
    ↳ status: "success"
    ↳ <prototype>: Object { ... }
```

Figura 5. Console do navegador ao requisitar a cesta de outro usuário: resposta 200 com um UserId que não corresponde à conta autenticada.

Controle de acesso quebrado na cesta

● CVSS 6.5 Médio

Uma conta autenticada lê a cesta de outra apenas trocando o identificador na URL. O servidor não verifica se o token pertence ao dono do recurso.

```

// Verifica se o token é válido e se o usuário é autenticado
function verificarToken(token) {
    // Verifica se o token é válido
    if (!token) {
        return false;
    }
    // Verifica se o token é válido
    if (!token) {
        return false;
    }
    // Verifica se o token é válido
    if (!token) {
        return false;
    }
    // Verifica se o token é válido
    if (!token) {
        return false;
    }

```

```

fetch("/rest/basket/2",
  headers: { Authorization: `Bearer ${token}` }
}).then(r=>r.json()).then(
  // deveria responder 403

```

A prova de conceito registra a cesta da primeira, sem depen

da 200 com um UserId que não

```

▼ Object { status: "success", data: { _ } }
  data: Object { id: 2, UserId: 2, createdAt: "2026-09-04T13:48:59.480Z", _ }
    ▼ Products: Array [ { _ } ]
      0: Object { id: 4, name: "Raspberry Juice (1000ml)", description: "Made from blended Raspberry Pi, water and sugar.", _ }
        length: 1
        <prototype>: Array []
      UserId: 2
      coupon: null
      createdAt: "2026-09-04T13:48:59.480Z"
      id: 2
      updatedAt: "2026-09-04T13:48:59.480Z"
    <prototype>: Object { _ }
    status: "success"
    <prototype>: Object { _ }

```

Figura 5. Console do navegador ao requisitar a cesta de outro usuário: resposta 200 com um UserId que não corresponde à conta autenticada.

O erro expõe detalhes do servidor

● CVSS 5.3 Médio · pontuação do relatório para exposição de informação

1. Entrada malformada → informação interna

```
GET /rest/basket/1 HTTP/1.1
Authorization: BearerSemEspaco
```

```
HTTP/1.1 401 Unauthorized
<h1>OWASP Juice Shop (Express ^4.22.1)</h1>
UnauthorizedError: Format is Authorization: Bearer [token]
```

Trechos selecionados; campos não exibidos omitidos.

[Requisição V4 · HTML de erro](#)

2. Resposta da página inicial

```
HTTP/1.1 200 OK
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
Content-Type: text/html; charset=UTF-8
```

Recorte do TXT. A ausência de CSP foi conferida na lista completa, não apenas neste recorte.

[Headers completos \(.txt\)](#)

O 401 está correto. O problema é a página pública revelar tecnologia e detalhes do erro. Isso facilita reconhecimento; não comprova uma invasão pelo Express.

CSP ausente: falta uma camada de contenção de scripts. **HSTS:** não é evidência de falha neste alvo HTTP por IP; é recomendação para implantação HTTPS com domínio. [MDN]

Por que V4 permanece? A05 está evidenciada. A06 pergunta outra coisa: existe uma vulnerabilidade conhecida aplicável ao componente? Essa investigação não confirmou A06 e não entra na contagem.



Figura 6. Erro verboso capturado no Postman. Clique para ampliar.



Figura 7. Cabeçalhos completos da página inicial. Clique para ampliar.

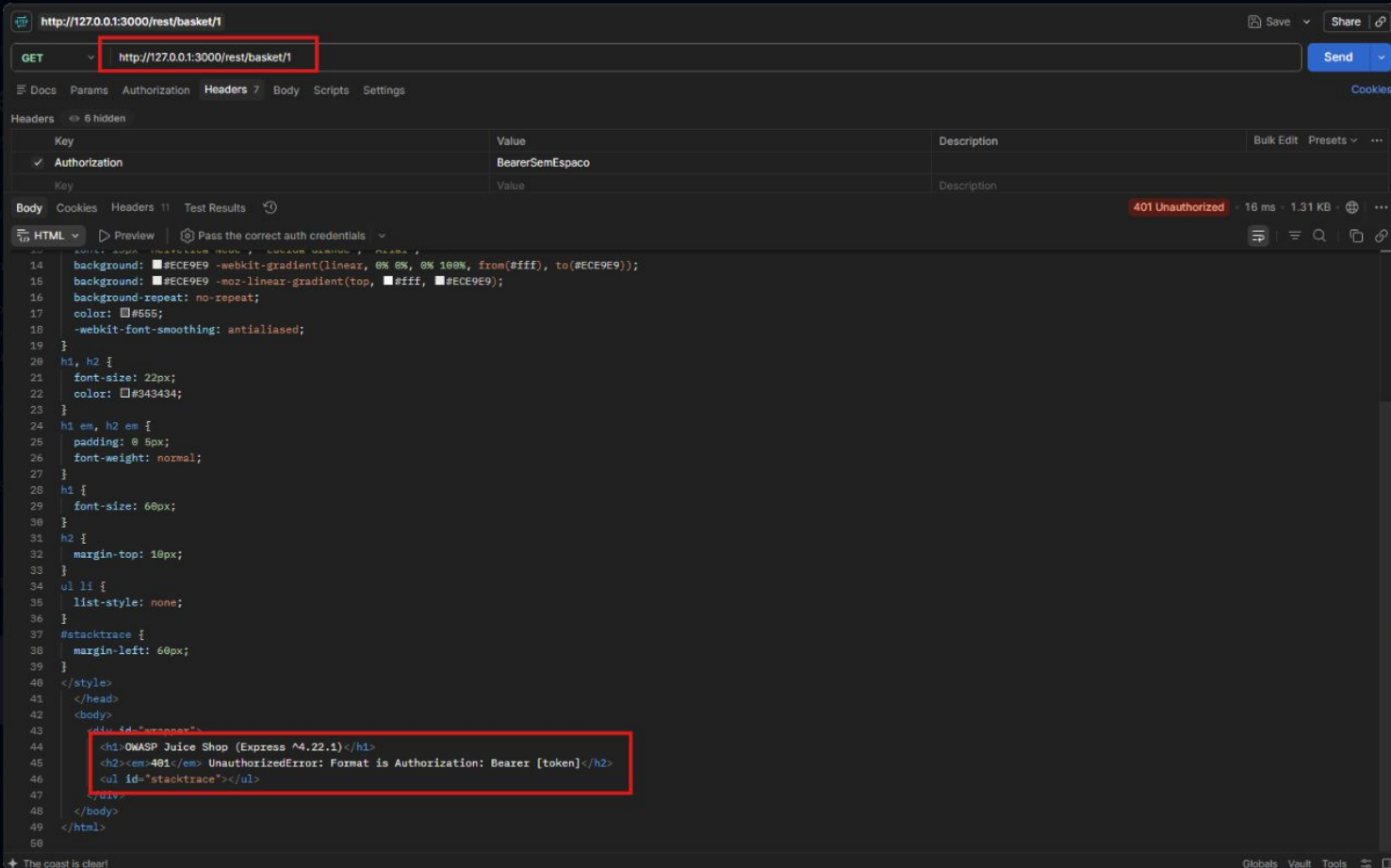


Figura 6. Erro verboso capturado no Postman. Clique para ampliar.

The screenshot shows a web browser's developer tools interface. The address bar displays `http://127.0.0.1:3000/`. The `GET` method is selected. The `Headers` tab is active, showing the response headers. The status is `200 OK` with a response time of `8 ms` and a size of `3.29 KB`.

Key	Value
Access-Control-Allow-Origin	*
X-Content-Type-Options	nosniff
X-Frame-Options	SAMEORIGIN
Feature-Policy	payment 'self'
X-Recruiting	/#/jobs
Accept-Ranges	bytes
Cache-Control	public, max-age=0
Last-Modified	Sat, 05 Sep 2026 17:32:40 GMT
ETag	W/"24b1-1a072a12b27"
Content-Type	text/html; charset=UTF-8
Vary	Accept-Encoding
Content-Encoding	br
Date	Sat, 05 Sep 2026 17:45:06 GMT
Connection	keep-alive
Keep-Alive	timeout=5
Transfer-Encoding	chunked

Figura 7. Cabeçalhos completos da página inicial. Clique para ampliar.

Índice público leva a documento confidencial

● CVSS 7.5 Alto · confidencialidade no cenário avaliado

1. O índice revela o nome do recurso

```
GET /ftp/ HTTP/1.1
Host: 127.0.0.1:3000
```

```
HTTP/1.1 200 OK
<a href="acquisitions.md" ...>...</a>
<a href="package.json.bak" ...>...</a>
```

Recorte da requisição, status e HTML; reticências abreviam atributos/conteúdo.
[Pedido sem Authorization](#) · [Listagem HTML](#)

A listagem preservada contém 10 arquivos e uma subpasta. Listado não significa necessariamente baixável.

2. Conteúdo efetivamente obtido

Planned Acquisitions

> This document is confidential! Do not distribute!

Our company plans to acquire several competitors within the next year.

Trecho inicial literal de acquisitions.md; o HTTP 200 do download aparece no print abaixo.
[Documento obtido](#)

O dano demonstrado é exposição de informação empresarial. Dados pessoais não foram identificados neste trecho.

Correção: retirar arquivos sensíveis da área pública e autorizar downloads. Desativar somente o índice deixa possível o acesso pela URL conhecida.



Figura 8. Índice de /ftp/ no Postman.



Figura 9. Download do documento confidencial no Postman.

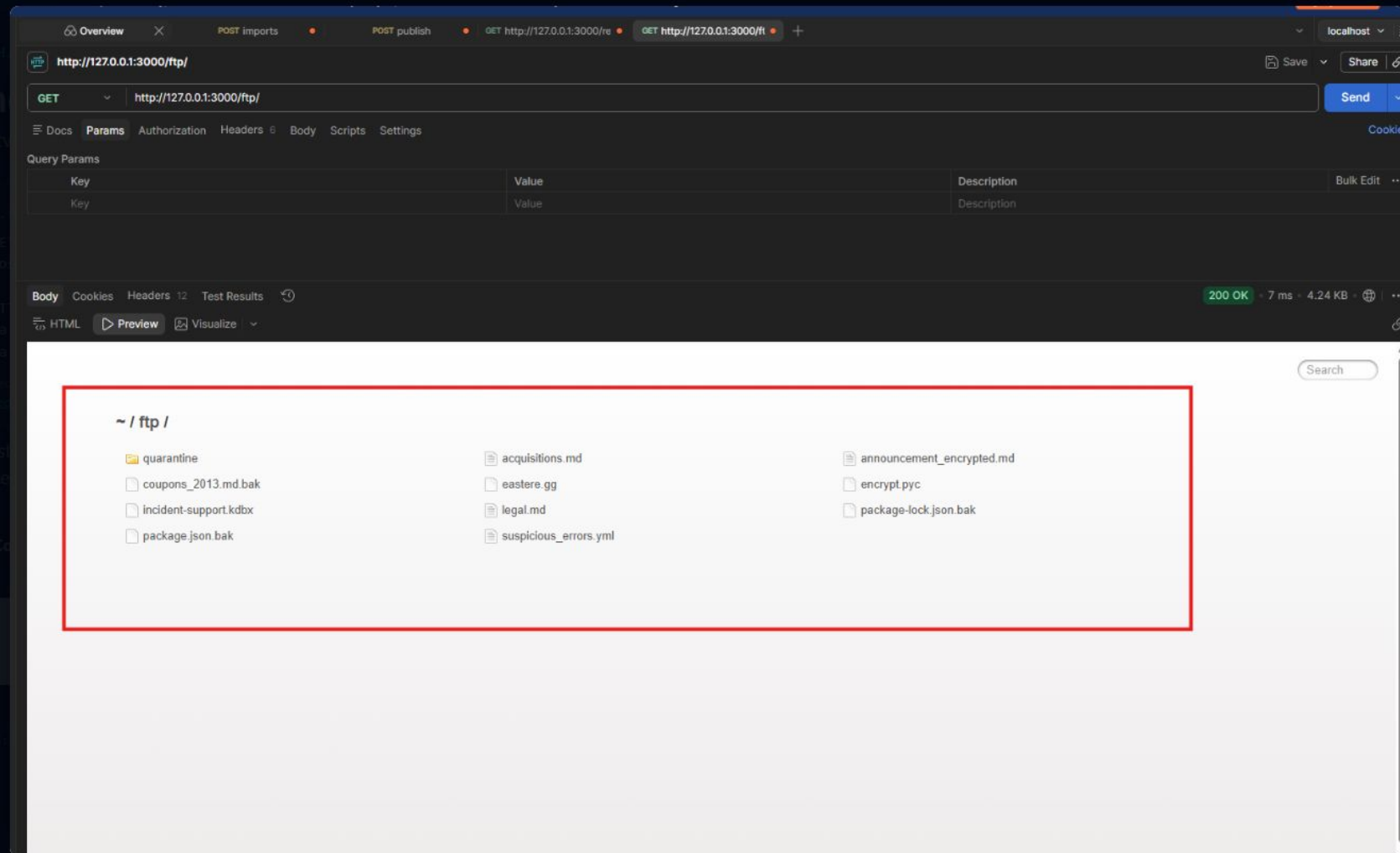


Figura 8. Índice de /ftp/ no Postman.

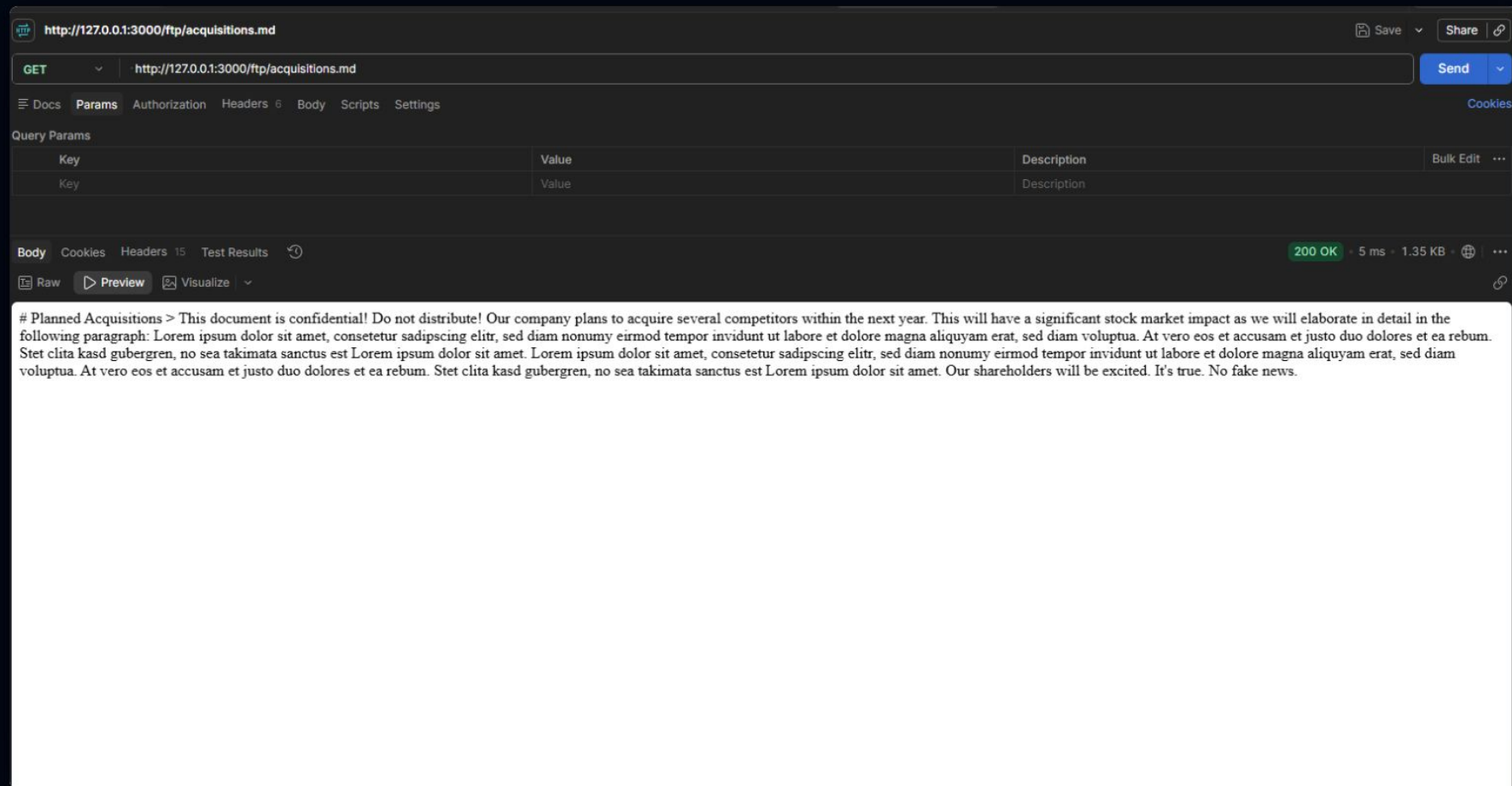


Figura 9. Download do documento confidencial no Postman.

Quinze erros não impediram o próximo login

● CVSS 6.5 Médio · pontuação adotada no relatório

Força bruta: repetir candidatos à senha de uma conta. O ensaio verificou se erros sucessivos acionariam uma restrição; a senha final já era conhecida pela equipe.

1. Mesma conta, senhas incorretas

```
POST http://127.0.0.1:3000/rest/user/login
{"email":"bruteforce-837d92427c@lab.local",
 "password":"senhaErrada1"}
```

HTTP 401 Unauthorized
Invalid email or password.

JSON reformatado. Arquivos 02–16: 15 senhas diferentes, todas HTTP 401.
[Pedido - 02](#) · [Primeiro erro - 02](#) · [Último erro - 16](#)

401: aquela senha foi rejeitada. Isso, sozinho, não informa se há limite para repetir o teste.

2. Senha correta logo após os erros

```
POST http://127.0.0.1:3000/rest/user/login
{"email":"bruteforce-837d92427c@lab.local",
 "password":"SenhaCorreta#9"}
```

HTTP 200 OK
Date: Sat, 05 Sep 2026 19:52:05 GMT

JSON reformatado. O corpo original contém authentication.token; token omitido da projeção.
[Pedido - 17](#) · [Login correto - 17](#)

200: a conta ainda aceitava autenticação válida depois dos 15 erros. Nenhum 403/429 foi observado.

Conclusão limitada ao ensaio: não houve bloqueio impeditivo após 15 falhas. Isso não prova tentativas infinitas, ausência de logs ou ausência de qualquer atraso. Os headers Date têm resolução de segundos.

17 pares = cadastro (01) + 15 erros (02–16) + acerto (17). Defesa: limite por conta, espera progressiva e MFA. [\[OWASP\]](#)

CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:N · impacto potencial; nenhuma senha desconhecida foi descoberta neste ensaio.

Resposta fraca permite redefinir a senha

● CVSS 9.1 Crítico · cenário de resposta previsível adotado no relatório

Conta de teste com resposta "John": pergunta consultável sem login → quatro erros → resposta correta → nova senha aceita.

1. Consulta e primeiro erro

```
GET /rest/user/security-question?email=...
HTTP 200 OK
"question": "Your eldest siblings middle name?"
```

```
POST /rest/user/reset-password
"answer": "Michael"
HTTP 401 Unauthorized
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 99
Wrong answer to security question.
```

Recortes de duas trocas HTTP; URL abreviada e campos JSON selecionados.
[Pergunta .04](#) · [Erro .05](#)

2. Quinta resposta e login final

```
POST /rest/user/reset-password
"answer": "John"
"new": "SenhaForjada#2"
HTTP 200 OK
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 95
```

```
POST /rest/user/login
"password": "SenhaForjada#2"
HTTP 200 OK
```

Recortes de duas trocas HTTP; os pedidos de reset também enviam email e repeat.
[Reset .09](#) · [Novo login .10](#)

Existe limite anunciado. Os headers contradizem "tentativas ilimitadas". A resposta fraca foi acertada antes de esgotá-lo; não foi testado o bloqueio na centésima tentativa.

A04: o fluxo confia numa resposta adivinhável para trocar a credencial. Correção: token imprevisível, de uso único e com expiração, além de controles de tentativa. [OWASP]

CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N · 9.1 é o valor do relatório; pré-condição: resposta previsível. Não extrapolar para qualquer conta.

Sete achados, seis categorias

Mapeamento adotado: A01, A02 e A03 obrigatórias, mais A04, A05 e A07.

Categoria	Prova principal
A01 - V3	Token B lê cesta A
A02 - V2	MD5 da senha no JWT
A03 - V1	SQLi concede sessão administrativa
A04 - V7	Resposta fraca → troca de senha
A05 - V4 e V5	Erro verboso e arquivo público
A07 - V6	Login válido após 15 falhas

Confirmação automatizada de A03

Parameter: JSON_email ((custom) POST)

Type: boolean-based blind

back-end DBMS: SQLite

Linhas selecionadas do log; não é uma varredura das seis categorias.

[Log do sqlmap \(txt\)](#)

Verificação independente de A02

Hash presente no campo 'password' do payload: 8b8c7a4e51e96bd9c5355af999eabf07

MD5 calculado localmente sobre a senha em texto claro:

8b8c7a4e51e96bd9c5355af999eabf07

Hashes idênticos: True

Trechos selecionados; campos não exibidos omitidos.

[Análise do JWT \(txt\)](#)

A06: investigação registrada sobre Express, sem achado confirmado; não comprova segurança de todas as dependências. **A08–A10:** não investigadas. O mínimo numérico está coberto; permanece a limitação de análise das demais categorias.

RECOMENDAÇÕES

Corrigir a causa e verificar o resultado

Ordem do relatório; as premissas do CVSS e o contexto do ativo precisam ser considerados.

Prioridade	Controle recomendado	O que retestar
V1 - 9.1	Consulta parametrizada	Payload não concede sessão
V7 - 9.1*	Token imprevisível, único e expirável	Pergunta fraca não redefine a senha
V5 - 7.5	Remover arquivo público / autorizar acesso	URL direta também é protegida
V2 - 6.5*	Retirar hash do JWT; hash de senha adequado	Token sem password
V3 - 6.5	Autorização por objeto no servidor	Conta B não recebe cesta A
V6 - 6.5*	Limites por conta, espera progressiva, MFA	Política de tentativas funciona
V4 - 5.3	Erro genérico e política CSP	Erro sem detalhe interno; CSP coerente

O controle atual de V7 não evitou o cenário testado

HTTP 200 OK

X-RateLimit-Limit: 100

X-RateLimit-Remaining: 95

Resultado do reset com John. Limitar volume não torna uma resposta previsível um fator forte.

[Reset - 09](#)

* Scores preservados do relatório, com ressalvas de cenário discutidas nos achados. Recomendações não foram implementadas/retestadas. HSTS: considerar em HTTPS com domínio.

SÍNTESE

A evidência define o alcance da conclusão

Falhas de consulta, acesso a dados e recuperação de conta no laboratório local.

V1 - entrada na consulta

O payload testado concedeu sessão administrativa, sem senha correta.

V3 - autorização por objeto

Conta comum leu a cesta de outra, apesar de autenticada com seu próprio token.

V7 - recuperação fraca

Resposta previsível permitiu redefinir a senha mesmo com limite anunciado.

V6: senha errada rejeitada

HTTP 401 Unauthorized
Invalid email or password.

Erro de credencial; sozinho não prova um bloqueio de tentativas.

[Primeiro erro - 02](#)

V7: resposta errada rejeitada

HTTP 401 Unauthorized
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 99
Wrong answer to security question.

Mesmo status 401, mas com contador anunciado. Os controles devem ser analisados por fluxo.

[Erro - 05](#)

Lição: guardar pedido, resposta e efeito permite revisar interpretações. V4 confirma A05; A06 não foi confirmada. V6 sustenta o ensaio de 15 erros; V7 sustenta o cenário de resposta fraca.

Próximos passos: aprofundar dependências e A08–A10, revisar as premissas dos scores e retestar as correções propostas. Obrigado!